Blatt 8 - Smart Pointers und OOP-Basics

Übungsblatt 7 wird sich mit dem implementieren einiger einfachen Klassen sowie 'unique_ptr' aus der 'std-library' beschäftigen.

Bei Fragen oder Problemen, gerne ins Moodle schreiben!

Exercise 1 — Eure eigenen Klassen

Zum ersten mal erzeugen Sie die 'Klassen' in Ihrem Projektordner selbst.
***Testen** Sie Ihren Code in `main.cpp`.*
Dazu sollten Sie mindestens ein `Rectangle` und `Circle` erzeugen und Beider Flächen berechnen.

a) Implementieren Sie eine `Klasse` namens `Rectangle`. Definieren Sie in der `Header-File` der `Klasse`die `privaten` Felder `width`und `height` mit passenden Datentypen. Schreiben Sie eine Methode `calculate_area`, die Ihrem Namen gerecht wird und die Fläche zurückgibt.
Die Fläche eines `Rectangle` ist das Produkt aus `width` und `height`.

b) Implementieren Sie eine `Klasse` namens `Circle`. Definieren Sie in der `Header-File` der `Klasse`das `private` Feld `radius` mit einem passenden Datentypen. Schreiben Sie eine Methode `calculate_area`, die Ihrem Namen gerecht wird und die Fläche zurückgibt.
Die Fläche eines `Circle` ist dessen Radius quadriert und mit Pi (Benutzen Sie 3.141) multipliziert.

c) Schreiben Sie `getter` und `setter` Methoden für Alle Felder der Beiden Klassen.

Exercise 2 — Smart Pointer

Testen Sie Ihren Code in der `main.cpp-Datei`.

a) Erzeugen Sie einen `std::unique_ptr`, der auf eine Variable mit dem Wert `42` zeigt. Erzeugen Sie einen weiteren `std::unique_ptr` und versuchen Sie diesem den Anderen Pointer zu übergeben. Beschwert sich der Compiler? Erklären Sie das Verhalten basierend auf Ihrem Verständnis von unique Pointern.

b) Benutzen Sie nun `std::move(<unique pointer>)`, um den ersten Pointer dem Zweiten zu zuweisen. Was passier mit dem ersten Pointer? Probieren Sie ein bisschen herum und erklären Sie Ihre Ergebnisse.
Informationen zu `unique_ptr` können Sie auf den Vorlesungsfolien oder im Internet finden.